

BMP180

Sensor de Presión y Temperatura BMP180 con ESP32-C6

Sistemas Embebidos

MicroPython · I2C · BMP180

1. Objetivo

El objetivo de esta práctica es integrar el sensor barométrico BMP180 al ESP32-C6 mediante el protocolo I2C, implementando una librería propia en MicroPython capaz de leer temperatura, presión atmosférica y calcular la altitud sobre el nivel del mar. Los conceptos reforzados son:

- Lectura de registros de calibración del sensor a través de I2C.
- Implementación del algoritmo de compensación definido en el datasheet del BMP180.
- Uso de propiedades Python (@property) para abstraer la lectura de magnitudes físicas.
- Cálculo de altitud mediante la fórmula barométrica estándar.
- Manejo de excepciones para detección de fallos de hardware en tiempo de ejecución.

1.1 Objetivos Específicos

- Configurar el bus I2C del ESP32-C6 y detectar el BMP180 en su dirección estándar 0x77.
- Leer e interpretar los 11 coeficientes de calibración almacenados en la memoria EEPROM del sensor (registros 0xAA–0xBF).
- Implementar el algoritmo de compensación de temperatura de dos etapas (cálculo de B5) para obtener valores en grados Celsius.
- Implementar el algoritmo de compensación de presión de seis parámetros intermedios (B3, B4, B6, B7) para obtener valores en Pascales.
- Calcular la altitud en metros usando la fórmula barométrica internacional con presión de referencia configurable.

1.2 Alcance de la Práctica

El alcance comprende el desarrollo en firmware con MicroPython sobre el ESP32-C6: lectura de calibración, compensación matemática de temperatura y presión, y cálculo de altitud. Se utiliza el modo de sobremuestreo OSS=0 (presión de baja resolución). No se cubren modos de alta precisión (OSS 1-3), almacenamiento de datos, ni transmisión inalámbrica de lecturas.

2. Hardware Utilizado

2.1 Componentes

Componente	Cantidad	Descripción
ESP32-C6	1	Microcontrolador principal, maestro I2C a 400 kHz

Sensor BMP180	1	Sensor barométrico digital: temperatura, presión y altitud vía I2C
Resistencias 4.7 kΩ	2	Pull-up para líneas SDA y SCL del bus I2C

2.2 Mapa de Pines

Señal	GPIO ESP32-C6	Pin BMP180	Descripción
SCL	GPIO 7	SCL	Señal de reloj I2C (400 kHz)
SDA	GPIO 6	SDA	Línea de datos bidireccional I2C
VCC	3.3 V	VIN	Alimentación del sensor
GND	GND	GND	Referencia común de tierra

2.3 Características del Sensor BMP180

Parámetro	Valor
Rango de presión	300 – 1100 hPa
Rango de temperatura	-40 °C a +85 °C
Resolución de presión	0.01 hPa (OSS=0) hasta 0.003 hPa (OSS=3)
Resolución de temperatura	0.1 °C
Interfaz	I2C, dirección fija 0x77
Voltaje de operación	1.8 V – 3.6 V
Coeficientes de calibración	11 valores en EEPROM interna (registros 0xAA – 0xBF)

2.4 Pruebas Eléctricas Realizadas

Antes de cargar el firmware se realizaron las siguientes verificaciones:

- **Voltaje de alimentación:** Se verificó que la salida de 3.3 V del ESP32-C6 cumpliera el rango operativo del BMP180 (1.8 V – 3.6 V). Valores fuera de rango dañan el sensor irreversiblemente.
- **Pull-up I2C:** Se midió ~4.7 kΩ entre SDA/SCL y 3.3 V. Resistencias menores a 1 kΩ causarían exceso de corriente; mayores a 10 kΩ degradan la señal a 400 kHz.
- **Detección en bus:** Se ejecutó `i2c.scan()` desde MicroPython y se confirmó la presencia del dispositivo en la dirección 0x77 antes de instanciar la clase BMP180.

- **Lectura de calibración:** Se verificó que los 22 bytes de calibración (0xAA–0xBF) devolvieran valores distintos de 0x00 y 0xFF, que indicarían un sensor defectuoso o mal conectado.

3. Arquitectura del Software

El proyecto se divide en dos archivos con responsabilidades claramente separadas:

Archivo	Responsabilidad
bmp180_library.py	Librería del sensor: carga de calibración, algoritmos de compensación de temperatura y presión, y cálculo de altitud.
BMP180_Ejemplo.py	Programa principal: inicialización del bus I2C, instancia del sensor, bucle de lectura y presentación de resultados por consola.

4. Descripción Técnica Detallada

4.1 Librería del Sensor — bmp180_library.py

Esta librería implementa la clase BMP180 que abstrae por completo la comunicación con el sensor. El usuario solo necesita llamar a las propiedades `temperature`, `pressure` y al método `get_altitude()`; toda la complejidad del protocolo y la compensación matemática queda encapsulada.

4.1.1 Constructor e Inicialización

El constructor recibe el objeto I2C ya inicializado, almacena la dirección del sensor (0x77, fija en hardware) y llama inmediatamente a la carga de calibración:

```
class BMP180:
    def __init__(self, i2c):
        self.i2c = i2c
        self.addr = 0x77 # Dirección I2C fija del BMP180
        self._load_calibration()
```

Si la comunicación falla en este punto (sensor no conectado, dirección incorrecta), se lanza una excepción que el código principal captura para evitar un programa en estado inválido.

4.1.2 Carga de Coeficientes de Calibración

El BMP180 almacena en su EEPROM interna 11 coeficientes únicos por unidad, fabricados durante el proceso de producción. Sin ellos, los valores de temperatura y presión serían incorrectos. Se leen 22 bytes a partir del registro 0xAA:

```
def _load_calibration(self):
    cal = self.i2c.readfrom_mem(self.addr, 0xAA, 22)
    self.AC1, self.AC2, self.AC3,    # Coef. de presión (con signo)
    self.AC4, self.AC5, self.AC6,    # Coef. de presión (sin signo)
    self.B1, self.B2,                # Coef. de presión (con signo)
    self.MB, self.MC, self.MD =      # Coef. de temperatura (con signo)
    ustruct.unpack(">hhhHHHhhhhh", cal)
```

El formato ">hhhHHHhhhhh" de ustruct indica: big-endian (>), tres enteros de 16 bits con signo (h = AC1, AC2, AC3), tres sin signo (H = AC4, AC5, AC6) y cinco con signo (h = B1, B2, MB, MC, MD).

4.1.3 Lectura y Compensación de Temperatura

La temperatura sigue un proceso de dos pasos: primero se dispara una medición en el sensor, luego se lee el valor crudo (UT) y se compensa con los coeficientes de calibración para obtener grados Celsius:

```
@property
def temperature(self):
    # 1. Ordenar medición de temperatura (comando 0x2E)
    self.i2c.writeto_mem(self.addr, 0xF4, b'\x2E')
    time.sleep(0.005)          # Esperar 4.5 ms mínimo
    # 2. Leer valor crudo UT (2 bytes, big-endian, con signo)
    ut = ustruct.unpack('>h', self.i2c.readfrom_mem(self.addr, 0xF6, 2))[0]
    # 3. Aplicar compensación (algoritmo del datasheet BMP180)
    x1 = (ut - self.AC6) * self.AC5 >> 15
    x2 = (self.MC << 11) // (x1 + self.MD)
    self.b5 = x1 + x2          # b5 se guarda para usar en presión
    return ((self.b5 + 8) >> 4) / 10.0  # Resultado en °C
```

El valor b5 es un parámetro intermedio compartido: la compensación de presión también lo necesita, por eso se guarda como atributo de instancia (self.b5).

Variable	Operación	Significado
x1	$(UT - AC6) * AC5 \gg 15$	Corrección lineal con coef. AC5 y AC6
x2	$(MC \ll 11) // (x1 + MD)$	Corrección no lineal con coef. MC y MD
b5	$x1 + x2$	Parámetro compartido con compensación de presión

T (°C)	$((b5 + 8) \gg 4) / 10.0$	Temperatura final compensada
--------	---------------------------	------------------------------

4.1.4 Lectura y Compensación de Presión

La presión requiere primero que b5 esté actualizado (por eso se llama self.temperature al inicio), luego se dispara la medición de presión y se aplica un algoritmo de compensación más extenso con seis variables intermedias:

```
@property
def pressure(self):
    self.temperature          # Actualizar b5
    self.i2c.writeto_mem(self.addr, 0xF4, b'\x34') # Cmd presión OSS=0
    time.sleep(0.005)
    up = self.i2c.readfrom_mem(self.addr, 0xF6, 3)
    up = ((up[0] << 16) | (up[1] << 8) | up[2]) >> 8 # 3 bytes → entero
    # --- Algoritmo de compensación (datasheet BMP180, sección 3.5) ---
    b6 = self.b5 - 4000
    x1 = (self.B2 * (b6 * b6 >> 12)) >> 11
    x2 = self.AC2 * b6 >> 11
    x3 = x1 + x2
    b3 = (((self.AC1 * 4 + x3) << 0) + 2) >> 2
    x1 = self.AC3 * b6 >> 13
    x2 = (self.B1 * (b6 * b6 >> 12)) >> 16
    x3 = ((x1 + x2) + 2) >> 2
    b4 = (self.AC4 * (x3 + 32768)) >> 15
    b7 = (up - b3) * (50000 >> 0)
    p = (b7 * 2) // b4
    x1 = (p >> 8) * (p >> 8)
    x1 = (x1 * 3038) >> 16
    x2 = (-7357 * p) >> 16
    return p + ((x1 + x2 + 3791) >> 4) # Resultado en Pascales
```

Var.	Propósito	Coeficientes usados
b6	Desviación de temperatura respecto a 4000	b5
b3	Compensación de offset de presión	AC1, AC2, B2
b4	Factor de escala de presión (unsigned)	AC3, AC4, B1
b7	Presión cruda escalada para el cálculo final	up, b3
p	Presión compensada con corrección de 2do orden	b7, b4, x1, x2

4.1.5 Cálculo de Altitud

La altitud se calcula a partir de la presión medida usando la fórmula barométrica internacional. Se acepta una presión de referencia configurable (por defecto 1013.25 hPa, presión al nivel del mar en condiciones estándar):

```
def get_altitude(self, baseline=1013.25):
    p_hpa = self.pressure / 100          # Convertir Pa → hPa
    return 44330 * (1 - (p_hpa / baseline) ** (1/5.255))
```

La constante **44330** representa la altura máxima teórica de la atmósfera en metros. El exponente **1/5.255** proviene de la relación entre presión y altitud en la atmósfera estándar ISO 2533. Si se conoce la presión local exacta al nivel del mar del lugar de medición, se puede pasar como argumento **baseline** para mayor precisión.

4.2 Programa Principal — BMP180_Ejemplo.py

El archivo de ejemplo muestra el patrón recomendado para usar la librería: inicialización segura con manejo de excepciones y bucle de lectura periódica.

4.2.1 Inicialización del Bus I2C y del Sensor

```
import machine
from bmp180_library import BMP180

i2c = machine.I2C(0, scl=machine.Pin(7), sda=machine.Pin(6))

try:
    sensor = BMP180(i2c)
    print("Sistema BMP180 activo.")
except Exception as e:
    print("Fallo de hardware:", e)
    sensor = None
```

El bloque try/except cumple una función crítica: si el sensor no está conectado o la dirección I2C no responde, `_load_calibration()` lanza una excepción. Sin este bloque, el programa fallaría sin un mensaje informativo. Al asignar `sensor = None` cuando falla, el bucle `while sensor` no se ejecuta, evitando errores en cascada.

4.2.2 Bucle de Lectura Periódica

```
while sensor:
    t = sensor.temperature          # Temperatura en °C
    p = sensor.pressure / 100       # Presión: Pa → hPa
    a = sensor.get_altitude()       # Altitud en metros (ref. 1013.25 hPa)
    print(f"T: {t:.1f}°C | P: {p:.1f}hPa | A: {a:.1f}m")
```

```
time.sleep(2) # Pausa de 2 segundos entre lecturas
```

La condición `while sensor` es equivalente a `while sensor is not None`: si la inicialización falló y `sensor` es `None`, el bucle no se ejecuta. La pausa de 2 segundos entre lecturas es suficiente para aplicaciones de monitoreo ambiental y evita sobrecargar el bus I2C.

Nota: cada iteración llama a **sensor.temperature** y luego a **sensor.pressure**. Internamente, `pressure` ya llama a `temperature` para actualizar `b5`, por lo que la temperatura se mide dos veces por iteración. Para optimizar, se puede guardar el valor de temperatura antes de leer la presión y evitar la doble medición.

5. Flujo General del Sistema

El siguiente diagrama describe la secuencia de ejecución completa desde el arranque:

```
INICIO → Importar machine, time, BMP180
↓
I2C(0, scl=GPIO7, sda=GPIO6, freq=400000)
↓
BMP180.__init__() → _load_calibration()
├── [Éxito] → leer 22 bytes (0xAA-0xBF) → guardar AC1..MD → sensor listo
└── [Fallo] → excepción → sensor = None → FIN (sin bucle)
↓
BUCLE while sensor:
├── sensor.temperature
│   ├── writeto(0xF4, 0x2E) → sleep 5ms → readfrom(0xF6, 2) → UT
│   └── compensar: x1, x2, b5 → T (°C)
├── sensor.pressure
│   ├── temperature() [actualizar b5] → writeto(0xF4, 0x34) → sleep 5ms
│   ├── readfrom(0xF6, 3) → UP → compensar: b6,b3,b4,b7,p → P (Pa)
├── sensor.get_altitude()
│   ├── P/100 → hPa → 44330*(1-(hPa/1013.25)^(1/5.255)) → A (m)
├── print(T, P, A)
└── sleep(2s) → repetir
```

6. Protocolo I2C — Comunicación con el BMP180

Toda la comunicación con el BMP180 ocurre a través del bus I2C. En esta práctica se utilizan dos tipos de operación:

Operación	Función MicroPython	Uso en el proyecto
Escritura en registro	<code>i2c.writeto_mem(addr, reg, data)</code>	Enviar comandos de medición (0x2E, 0x34) al registro 0xF4

Lectura de registro	i2c.readfrom_mem(addr, reg, n)	Leer calibración (0xAA, 22 bytes) y resultados (0xF6, 2-3 bytes)
---------------------	--------------------------------	--

El BMP180 requiere un retardo mínimo de 4.5 ms entre el comando de medición y la lectura del resultado (tiempo de conversión en modo OSS=0). En la librería este retardo se implementa con `time.sleep(0.005)` (5 ms, con margen de seguridad).

7. Dependencias y Bibliotecas

Módulo	Origen	Uso en el proyecto
machine.I2C	MicroPython stdlib	Bus I2C maestro (scan, read, write)
machine.Pin	MicroPython stdlib	Configuración de pines GPIO 6 y 7
time	MicroPython stdlib	<code>sleep()</code> para tiempos de conversión del sensor
ustruct	MicroPython stdlib	Decodificación de bytes en enteros con/sin signo big-endian
bmp180_library	Proyecto propio	Clase BMP180: calibración, compensación y altitud

8. Conclusiones

Esta práctica integra el sensor BMP180 al ecosistema ESP32-C6 + MicroPython, consolidando los siguientes aprendizajes:

- La calibración individual por unidad es fundamental en sensores MEMS: sin los coeficientes almacenados en la EEPROM del BMP180, los valores de temperatura y presión serían incorrectos para ese sensor específico.
- El algoritmo de compensación del BMP180 involucra aritmética entera de precisión variable (shifts de hasta 16 bits). La implementación fiel al datasheet es indispensable para obtener resultados correctos.
- El parámetro compartido b5 entre temperatura y presión exige que la temperatura siempre se mida antes de la presión, lo que se garantiza llamando a `self.temperature` dentro de `pressure`.
- El uso de `@property` en Python/MicroPython ofrece una interfaz limpia: `sensor.temperature` se lee como un atributo pero ejecuta toda la secuencia I2C internamente, ocultando la complejidad al usuario de la librería.
- El manejo de excepciones en la inicialización es una buena práctica en sistemas embebidos: permite detectar fallos de hardware de forma controlada sin bloquear el programa.

- La fórmula barométrica es sensible a la presión de referencia (baseline). Para mayor exactitud se recomienda calibrar con la presión local real al nivel del mar, especialmente en zonas de altitud elevada.